

MODERN JAVA (JDK 21 - 25) STUDY FLASHCARDS DECK

30 High-Impact Technical Flashcards for Senior & Lead Developers

This study companion is optimized for active recall prep. Each card covers critical architectural transitions, Project Loom concurrency, Leyden warmup, Amber ergonomics, and memory optimizations from JDK 21 to JDK 25.

Target Audience:	Senior / Staff Engineers, Lead Architects, Tech Leads
Core Focus:	LTS Releases (JDK 21 & JDK 25) & intermediate upgrades
Structure:	Front Side (Active recall scenario) & Back Side (Deep-dive explanation)
Themes:	Project Loom (Concurrency), Amber (Language), Leyden (AOT), GC Engines

CARD 1 OF 30

Front (Question):

What is "Carrier Thread Pinning" in Virtual Threads, and how is it resolved in Java 25?

Back (Explanation):

- **The Problem:** In JDK 21, when a virtual thread executing inside a **synchronized** block/method or invoking a native frame (JNI) performs a blocking blocking I/O operation, it cannot yield its carrier thread. It "pins" the underlying OS platform thread, stalling scalability.
- **The Resolution (JEP 491):** Java 24/25 internally overhauls monitor locks. Now, when a virtual thread encounters blocking operations inside a **synchronized** block, it safely unmounts from its carrier thread, releasing it to work-steal other virtual threads. JNI pinning is the only remaining pinning condition.

— — —

CARD 2 OF 30

Front (Question):

How do Scoped Values (JEP 506) solve the memory-leak and scalability issues of `ThreadLocal`?

Back (Explanation):

- **ThreadLocal Overhead:** `ThreadLocal` variables are mutable, meaning any thread can call `set()`. To prevent memory leaks, developers must meticulously call `remove()` in a `finally` block, which is error-prone in thread-pooled environments. Additionally, virtual threads multiply by millions; inheriting large `ThreadLocal` maps creates immense memory footprints.
- **Scoped Value Solution:** Scoped Values are strictly **immutable** and **bound to a execution scope**. Once bound via `ScopedValue.where(KEY, value).run(...)`, they cannot be changed. When the scope block exits, the binding is automatically discarded, neutralizing memory leaks without manual cleanup. Memory is shared with child threads via zero-overhead pointer sharing.

CARD 3 OF 30

Front (Question):

Explain the execution lifecycle of a custom Stream Gatherer (JEP 485).

=====

Back (Explanation):

**

Custom Gatherers represent a stateful intermediate stream operation defined by four functional components:

1. **Initializer (Supplier)**: Creates the mutable state holder (e.g., `() -> new ArrayList()`).
2. **Integrator (Integrator)**: Processes each incoming element. It takes `(state, element, downstream)` and returns a `boolean` (returning `false` signals immediate short-circuiting).
3. **Combiner (BinaryOperator)**: Defines how to merge two states during parallel evaluation of split stream chunks.
4. **Finisher (BiConsumer)**: Flushes any remaining cached state to downstream after the input source is exhausted.

CARD 4 OF 30

Front (Question):

What is the difference between `Gatherers.windowFixed` and `Gatherers.windowSliding`?

=====

Back (Explanation):

**

- **`windowFixed(size)`**: Groups elements into consecutive, non-overlapping blocks of a fixed size. For stream `[1, 2, 3, 4, 5]` and size `2`, it emits: `[1, 2]`, `[3, 4]`, and `[5]` (remaining trailing element in finisher).
- **`windowSliding(size)`**: Groups elements into overlapping windows of a fixed size, advancing by exactly one element per step. For stream `[1, 2, 3, 4]` and size `3`, it emits: `[1, 2, 3]` followed by `[2, 3, 4]`. Excellent for running averages or anomaly interval checks.

CARD 5 OF 30

Front (Question):

What is a "Prologue" in JEP 513 (Flexible Constructor Bodies) and what are its strict limits?

Back (Explanation):

**

- **Definition:** The prologue is the set of statements executed in a child constructor **before** calling **super()** or **this()**.
- **Allowed Operations:** Parameter validation (e.g., throwing **IllegalArgumentException**), calculating super arguments, or printing diagnostic logs.
- **Forbidden Operations:** You **cannot** read or write instance fields of the class being created or invoke instance methods during the prologue. **this** only becomes a valid reference **after** the parent constructor finishes, preventing uninitialized state leaks.

CARD 6 OF 30

Front (Question):

How does the JVM run a "Compact Source File" (JEP 512) without an explicit class declaration?

Back (Explanation):

**

- **Compilation Mechanics:** When **javac** compiles a compact source file (e.g., containing only **void main()**), it implicitly generates a top-level, **final** class residing in the unnamed package.
- **API Imports:** The compiler automatically imports the entire **java.base** module on-demand (equivalent to **import module java.base;**), making collections, math, and streams instantly accessible.
- **Entry Resolution:** The JVM launcher instantiates an instance of this implicit class using a default no-argument constructor and runs **main()**.

CARD 7 OF 30

Front (Question):

What is a Module Import Declaration (JEP 511), and how does it handle class name conflicts?

**Back (Explanation):**

**

- **Usage:** `import module M;` imports all public top-level classes and interfaces from all packages exported by module `M` (including transitive module dependencies). For example, `import module java.base;` imports lists, maps, streams, and files.
- **Name Conflicts:** If you import two modules containing classes of the same simple name (e.g., `java.util.Date` from `java.base` and `java.sql.Date` from `java.sql`), the compiler throws an ambiguous import error. You resolve this by declaring a single explicit import statement (e.g., `import java.sql.Date;`) which shadows the module-imported names.

CARD 8 OF 30

Front (Question):

How do Stable Values / Lazy Constants (JEP 502/526) improve performance compared to standard final fields?

**Back (Explanation):**

**

- **Limitation of Final:** A `final` field must be set in the constructor. If you need lazy-initialization (e.g., database connection loaded on demand), you must use synchronized double-checked locking, which prevents the JIT compiler from optimizing the field as a constant.
- **Stable Value Solution:** Declared via `StableValue.of()`, it supports thread-safe, deferred initialization exactly once. Once set via `orElseSet()`, the JVM trusts that the value is immutable and performs constant folding, compiling the lazy reference directly into machine-optimized constant code.

CARD 9 OF 30

Front (Question):**Explain "Strong Encapsulation" in modern JDKs and the flags used to bypass it.**

=====

Back (Explanation):

**

- **Encapsulation:** Since JDK 17, JVM internal classes (e.g., `sun.misc.*` or packages inside `java.base`) are strongly encapsulated. Attempts to access them via reflection (`setAccessible(true)`) throw an **`InaccessibleObjectException`**.

- **Bypass Flags:**

- `--add-exports module/package=target-module`: Exports a package at compile/run-time, making public types accessible.
- `--add-opens module/package=target-module`: Opens a package for deep reflection, allowing access to non-public fields and methods.

CARD 10 OF 30

Front (Question):**What are Compact Object Headers (JEP 519) and how do they save heap space?**

=====

Back (Explanation):

**

- **The Header:** Traditionally, a 64-bit JVM object has a header consisting of a **Mark Word** (64 bits) and a **Klass Word** (64 bits, or 32 bits with compressed pointers), using 96 to 128 bits of metadata.
- **The Compression:** Compact Object Headers compress the Klass pointer and metadata directly into the Mark Word. This slashes the object header down to exactly **64 bits (8 bytes)**, saving up to 22% of total heap memory and improving CPU L1/L2 cache line alignment.

CARD 11 OF 30

Front (Question):

Why does the compiler warn about "Unsafe" memory access methods in JEP 471, and what is the target migration path?

Back (Explanation):

**

- **The Risk:** `sun.misc.Unsafe` bypasses JVM safety boundaries, allowing direct memory manipulation. It is highly unstable, prone to segmentation faults, and prevents forward compatibility.
- **The Migration:** JDK 25 deprecates all memory-access methods in **Unsafe** for removal. Developers must migrate to the standardized **Foreign Function and Memory (FFM) API (JEP 454)**, which delivers safe, structured off-heap memory segments with equivalent bare-metal performance.

CARD 12 OF 30

Front (Question):

How does Project Leyden's Ahead-of-Time (AOT) Caching (JEP 483) accelerate startup?

Back (Explanation):

**

- **The Bottleneck:** Traditional JVM startup is bogged down by reading class files from JARs, parsing bytecodes, running verification, and dynamically linking symbolic references.
- **The AOT Cache:** By performing a training run, Project Leyden pre-loads and pre-links all classes used by the application into a compiled `.aot` cache. In subsequent production runs, the JVM memory-maps the entire linked class sub-graph instantly, bypassing verification and linking overhead.

CARD 13 OF 30

Front (Question):**What is the "One-Shot" AOT cache creation option in Java 25 (JEP 514)?****Back (Explanation):**

**

- **Traditional creation:** In JDK 24, creating an AOT cache required two separate command runs—one in `-XX:AOTMode=record` to generate a configuration, and a second in `-XX:AOTMode=create` to dump the cache file.
- **One-Shot creation:** Java 25 introduces `-XX:AOTCacheOutput=app.aot`. This single option handles the entire pipeline: it executes the training run, profiles class metadata, links references, and writes the finalized cache in one automated invocation.

CARD 14 OF 30

Front (Question):**How does JEP 515 (AOT Method Profiling) improve Java's time-to-peak performance (warmup)?****Back (Explanation):**

**

- **Cold Start JIT:** On startup, a JIT compiler must interpret methods thousands of times to collect execution profiles before it compiles them to highly optimized native code.
- **Leyden Profile Cache:** JEP 515 records method execution profiles during the training run and embeds them inside the AOT cache file. On production startup, the JIT compiler instantly reads these pre-recorded profiles and compiles hot methods to native code immediately, skipping the interpretation warmup phase entirely.

CARD 15 OF 30

Front (Question):**What is Generational ZGC, and why is the old non-generational mode removed in Java 25?**

Back (Explanation):

**

- **Weak Generational Hypothesis:** The fundamental rule that "most created objects die young".
- **Generational ZGC (JEP 439):** Separates the heap into young and old generations. It collects the young generation concurrently and extremely fast, avoiding the CPU overhead of scanning the entire heap.
- **Removal of Non-Gen (JEP 490):** Retaining both non-generational and generational codebases created immense JVM maintenance complexity. Since Generational ZGC delivers superior throughput and ultra-low latency simultaneously, the non-generational mode was removed in JDK 25.

CARD 16 OF 30

Front (Question):**Explain how Generational Shenandoah (JEP 521) differs from standard Shenandoah GC.**

Back (Explanation):

**

- **Standard Shenandoah:** An ultra-low latency, concurrent compaction collector that processes the entire heap as a single space.
- **Generational Shenandoah:** Integrates young and old generation spaces. It focuses intense, low-cost collection cycles on the young gen where most objects die, and runs less-frequent old gen cycles. This boosts overall application throughput and reduces memory allocation stalls under high-allocation workloads.

CARD 17 OF 30

Front (Question):**What is "Structured Concurrency" (JEP 505) and how does it prevent thread leaks?****Back (Explanation):**

**

- **The Problem:** Traditional concurrent task APIs (like `CompletableFuture` or `ExecutorService`) allow subtasks to run detached. If a parent task is cancelled, child threads continue running silently in the background, wasting CPU and leaking resources.
- **Structured Solution:** Enforces a lexical scope boundary using `try-with-resources` on `StructuredTaskScope`. All spawned subtask threads (`scope.fork()`) are logically grouped. If a subtask fails or the parent scope is interrupted, the JVM propagates cancellation down to all active child threads automatically.

CARD 18 OF 30

Front (Question):**What are `StructuredTaskScope`.Joiner policies and how do they orchestrate concurrency?****Back (Explanation):**

**

Joiners define the completion and cancellation criteria of structured task scopes:

- **ShutdownOnSuccess (or firstComplete):** Captures the result of the **first** successfully completed subtask and immediately cancels all other pending subtasks to save resources.
- **ShutdownOnFailure (or allSuccessfulOrThrow):** Joins all subtasks, but immediately shuts down the scope and propagates an exception the moment **any** subtask fails.

CARD 19 OF 30

Front (Question):

How do Unnamed Variables (JEP 456) improve code quality, and what is their syntax?

Back (Explanation):

- **The Problem:** Many Java structures force you to declare variables you never use (e.g., caught exceptions, unused lambda arguments, or destructured record elements).
- **The Syntax:** Denoted by a single underscore `_`.
- **Examples:**
 - `catch (NumberFormatException _)` (ignores the exception name)
 - `stream.collect(Collectors.toMap(String::toUpperCase, _ -> "EMPTY"))` (ignores the lambda parameter)
 - `for (Order _ : orders) { count++; }` (ignores the loop variable)

...

CARD 20 OF 30

Front (Question):

What is "Record Pattern Matching" and how do you destructure records in switch statements?

Back (Explanation):

- **Definition:** Allows you to test if an object matches a specific Record type and destructure its fields directly in a single statement.
- **Code Example:**

```
`java`  
if (obj instanceof Point(int x, int y)) {  
    System.out.println("Coordinates: " + x + ", " + y);  
}
```

This completely eliminates the boilerplate of casting the object and invoking getter methods.

— — —

CARD 21 OF 30

Front (Question):

Explain JEP 507 (Primitive Types in Pattern Matching, instanceof, and switch).

Back (Explanation):

**

- **The Limitation:** Prior to JDK 25, pattern matching and the `instanceof` operator were strictly restricted to reference types (objects). You could not write `if (val instanceof int i)`.
- **The Feature:** Expands pattern matching to primitive types. You can now use `instanceof` with primitives to perform safe type casting and validation in switch cases (e.g., checking if a `double` can safely be cast to an `int` without loss of precision).

CARD 22 OF 30

Front (Question):

How do "Helpful NullPointerExceptions" (JEP 358) identify the root cause of chained crashes?

Back (Explanation):

**

- **Chained Calls:** In older JDKs, a line like `city = user.getAddress().getCity().getName()` throwing an NPE left you guessing which method returned null (`user`, `getAddress()`, or `getCity()`).
- **Helpful NPE:** The JVM analyzes class file bytecode dynamically at the crash site. It outputs a precise, human-readable error message: `Cannot invoke "City.getName()" because the return value of "Address.getCity()" is null`.

CARD 23 OF 30

Front (Question):

Explain the Foreign Function and Memory (FFM) API (JEP 454) and its safety model.

Back (Explanation):

- **Core Components:**

- **MemorySegment**: Represents a continuous region of memory (off-heap or on-heap).
- **Arena**: Orchestrates the lifecycle and thread safety of memory allocations.
- **Linker**: Connects Java code with native libraries (downcalls) and vice versa (upcalls).
- **Safety**: It isolates memory access to managed **Arena** scopes. Unaligned or out-of-bounds accesses trigger safe Java exceptions rather than native segmentation faults.

CARD 24 OF 30

Front (Question):

What is the difference between a Confined Arena and a Shared Arena in the FFM API?

Back (Explanation):

- **Confined Arena (`Arena.ofConfined()`):** Bound strictly to a single thread. Memory allocations can only be accessed or closed by the owning thread. Delivers maximum performance due to zero synchronization overhead.
- **Shared Arena (`Arena.ofShared()`):** Thread-safe. Allows multiple concurrent threads to read, write, and access memory segments. Closing the arena is safely coordinated across threads.

CARD 25 OF 30

Front (Question):**Explain how to use `try-with-resources` with `Inflater` and `Deflater` in Java 25.****Back (Explanation):**

**

- **The Improvement:** Historically, `java.util.zip.Inflater` and `Deflater` allocated off-heap native memory. Failing to manually call `end()` resulted in severe native memory leaks.
- **The Update:** In JDK 25, both classes implement `AutoCloseable`. You can now safely wrap them inside a `try-with-resources` statement:

```
`java
try (var deflater = new Deflater()) {
    deflater.setInput(data);
    // ... zip logic ...
} // close() is called automatically, releasing native memory safely
`
```

CARD 26 OF 30

Front (Question):**What is the new Java 25 standard system property `stdin.encoding`?****Back (Explanation):**

**

- **The Property:** `stdin.encoding` defines the recommended character encoding (Charset) to read data from `System.in` (console input).
- **The Purpose:** Decouples console input reading from the global `file.encoding` property, adapting dynamically to the host Operating System's active terminal encoding configuration. Can be overridden using `-Dstdin.encoding=UTF-8`.

CARD 27 OF 30

Front (Question):

Explain JEP 467 (Markdown Documentation Comments) and its benefits.

Back (Explanation):

- **The Syntax:** Uses triple slashes (`///`) instead of traditional `/** ... */` syntax.
- **The Benefit:** Allows writing Javadoc comments in Markdown. Instead of ugly, hard-to-read HTML tags (like `<pre>`, `<code>`, `<b>`), you can write clean Markdown with backticks, bullet lists, bold text, and code fragments. It is highly readable in both raw source form and compiled HTML.

— — —

CARD 28 OF 30

Front (Question):

What enhanced JAR validation checks are introduced in JDK 25?

Back (Explanation):

The **jar -validate** utility has been upgraded to scan archives and flag critical structural errors, including:

- **Duplicate entries:** Identifies file duplicates inside the archive.
- **Forbidden paths:** Flags paths containing leading slashes, backslashes (\), or drive/device letters.
- **Relative navigation:** Warns about dangerous path traversal patterns like `.` or `...`
- **Header ordering:** Detects discrepancies between Local File Headers (LOC) and Central Directory Headers (CEN).

CARD 29 OF 30

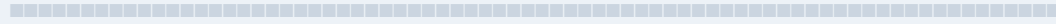
Front (Question):

What is "Region Pinning" in the G1 Garbage Collector (JEP 423)?

Back (Explanation):

- **The Issue:** Under JNI, passing a Java object to a native C function requires blocking GC to prevent the object from being relocated, stalling all thread executions.
- **The Solution:** Region Pinning allows G1 to continue garbage collection concurrently. Instead of stopping the entire GC, G1 only "pins" the specific memory regions containing the active JNI objects and continues cleaning the rest of the heap freely, reducing pause spikes in JNI-heavy environments.

CARD 30 OF 30

Front (Question):**What post-quantum cryptography features are integrated natively in JDK 25?****Back (Explanation):**

**

JDK 25 integrates standardized, lattice-based algorithms natively into the JVM security packages to protect against quantum computing attacks:

- **ML-KEM (JEP 496):** Module-Lattice-Based Key Encapsulation Mechanism, used to secure symmetric key exchange over insecure channels.
- **ML-DSA (JEP 497):** Module-Lattice-Based Digital Signature Algorithm, providing highly secure, quantum-resistant digital authentication and integrity verification.